23BCS10482 Shiva Gupta 12RGIDB

2) Given three integers n, a, b return n^{th} magical no. Since the ans may be large return 10^9+7.

A positive integer is magical if number is divisible by either a or b.

n = 1 b = 3 a = 2	n = 4 a = 2 b = 3
ans = 2	ans = 6

2s) Approach:

1) we take an empty array
2) We take initial counter
3) We push the min of a, b to array
4) After that till array size reaches n, we keep checking if min(a,b) + i the element is divisible by either a or b
5) If yes we push to result array
6) When the size of result array reaches n, we return last element

Dry Run:

1.) Take res array = {3}
2) Push min (a,b) = {2 3}
3) Start while loop till n = 4

2+1 % 2 or 3 → yes → push {2, 3}
2+2 % 2 or 3 → yes → Push {2, 3, 4}
2+3 % 2 or 3 → ~~yes~~ No → {2, 3, 4}
2+4 % 2 or 3 → Yes → {2, 3, 4, 6} (N reached)

4) Return last element of array
5.) We have the final answer

→ Complexity $O(n^{th}$ no.$)$ - min (a, b)
→ SC (O N)

To optimise this we need approach of complexity less than or equal to (gcd min (a, b))
so we use Binary Search

Approach:

1) Find gcd (a,b)
2) Use gcd to find LCM of a,b
3) Run binary search as 1 to n × min (a,b)
4) Calculate mid and for mid calculate count

$$count = \frac{mid}{a} + \frac{mid}{b} - \frac{mid}{lcm(a,b)}$$

5) If the required count is less increase search space by low = mid + 1
else high = mid - 1
6) If ans is big calculate using mod 1e9+7

Time complexity : O(log min (a,b))
SC : O(max min (a,b))

<u>Dry Run</u>
a = 2 b = 3 n = 4

1) GCD = 2
2) LCM = 6
3) Binary search b/w 1 to 2×4=8
4) Calc mid = (8-1)+1 /2 ⇒ 4
5) Calc count = $\frac{4}{2} + \frac{4}{3} - \frac{4}{6}$ ⇒ 2+1-0 ⇒ 3
6) Since it is less than n increase space
∴ low = 5
7) Calc mid ⇒ 6
count = $\frac{6}{2} + \frac{6}{3} - \frac{6}{6}$ ⇒ 3+2-1 = $\boxed{4}$

Since count = n required return mid

$\boxed{Ans = 6}$